

C5: Generative Adversarial Networks

Marina Rosell Murillo

NIU: 1779181

marina.rosellmurillo@autonoma.cat

Gerard Asbert Marcos

NIU: 1603295

gerard.asbert@autonoma.cat



Task 1

Task statement

Question:

Imagine you have a dataset X that you wish to generate new samples from.

- Why you would choose a GAN over any other type of generative network for this task?
- Examples:
 - If your data is extremely multimodal, would a GAN be the right choice?
 - What if sampling speed was a major constraint?
 - Or if there are concerns regarding the data privacy once you have a trained model?
 - Let your imagination run wild.

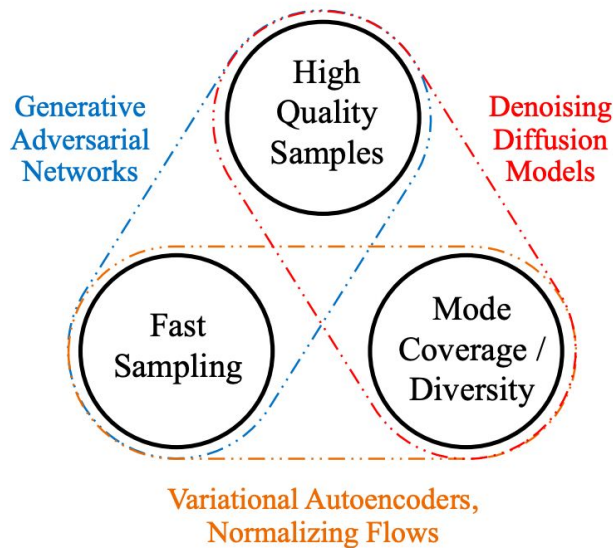
Generation Trilemma

To answer this question we have to take into account the major **strengths** and **weaknesses** of the GAN architecture.

For this it is important to mention the **generative trilemma**, which says that no generative model has these three positive attributes at the same time: **Image quality**, **mode coverage**, and **fast sampling**. The GAN specifically has quick sampling and high image quality, but it has a **very limited mode coverage**.

One of the reasons for this latter characteristic is that once the generator has found a set of generated images that fool the discriminator, it sticks to that set, therefore resulting in low variance in the generation (**mode collapse**).

Now, to respond that first question, one should choose a GAN over other generative networks when the task at hand requires **fast inference or/and high quality imaging**. Nevertheless if we need a certain amount of variance, one should pick another more explorative model, such as a Variational Autoencoder.



Other questions

Again, to answer the **next questions** and referencing the aforementioned characteristics of the GAN:

- **(Multimodal data?)** A GAN would be a bad choice for extremely multimodal data given the mode collapse problem mentioned before. It would result in increases instability as we add more data types.
- **(Sampling speed?)** Sampling speed is one of the strengths of the GAN, so a GAN would be a good choice for that type of problem.
- **(Data privacy?)** Given the fact that GANs are very prone to overfitting and memorization of the training data, it would not be the best choice if we have privacy constraints, given in some cases it would just replicate the training data in the output.
- **(Other concerns?)** Another important characteristic of GANs is that they are incredibly data hungry. That means that in our project we have a very small dataset, we should choose another type of generative model. Other possible constraints include controllability, as if we need to be very precise on what exactly we want the model to generate, then the GAN is not the best choice.

In conclusion, we want to use a GAN if the task at hand **requires fast sampling time** and **high image quality**, but should use another generative model if the task requires a model that **doesn't memorize** the input, **high variability**, **large datasets**, or **controllability** up to a certain point.

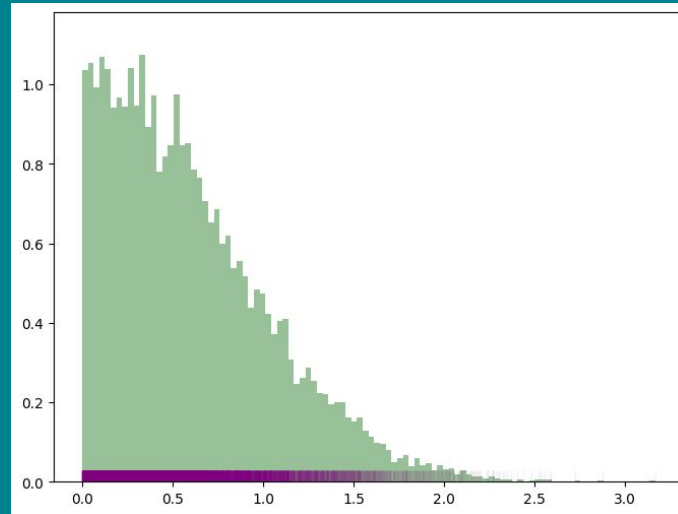
Task 2

Task statement

Question:

- Clearly, the training we've done above has failed to replicate the data (for the Normal distribution). But we're not interested in this easy dataset, we are interested in one of the harder distributions found below (`Laplacian`, `HalfNormal`, `PetitPrince`). **Choose one**, use the generated data found below each, and **determine the set of parameters and hyperparameters** that will make it easier for our network to **approximate the data**.

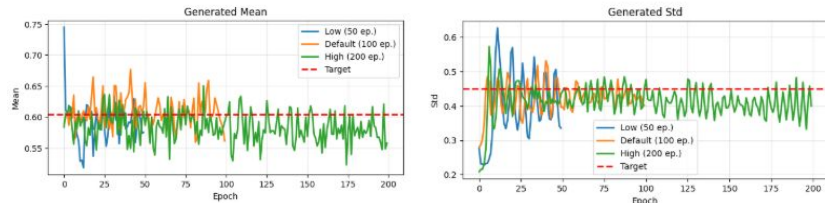
HalfNormal Distribution



Hyperparameter Analysis

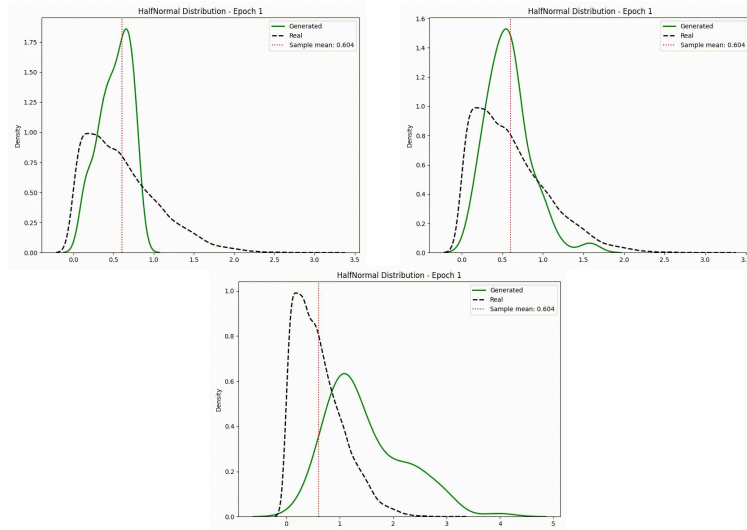
Number of epochs

- More epochs allow for better convergence, up to a point, as after passing a certain threshold of epochs, the estimated mean and std dont improve.



Latent Dimension

- A very small latent space ($z=2$) limits the generator, while a very big latent space ($z=15$) kind of overfits. This can be seen clearly in the animation where with a big latent space it tries to replicate the small curve at the top while overlooking the overall form. The default ($z=5$) doesn't fit the small curve at the top but fits the overall form. This is represented very well in the graph with the mean of each model.



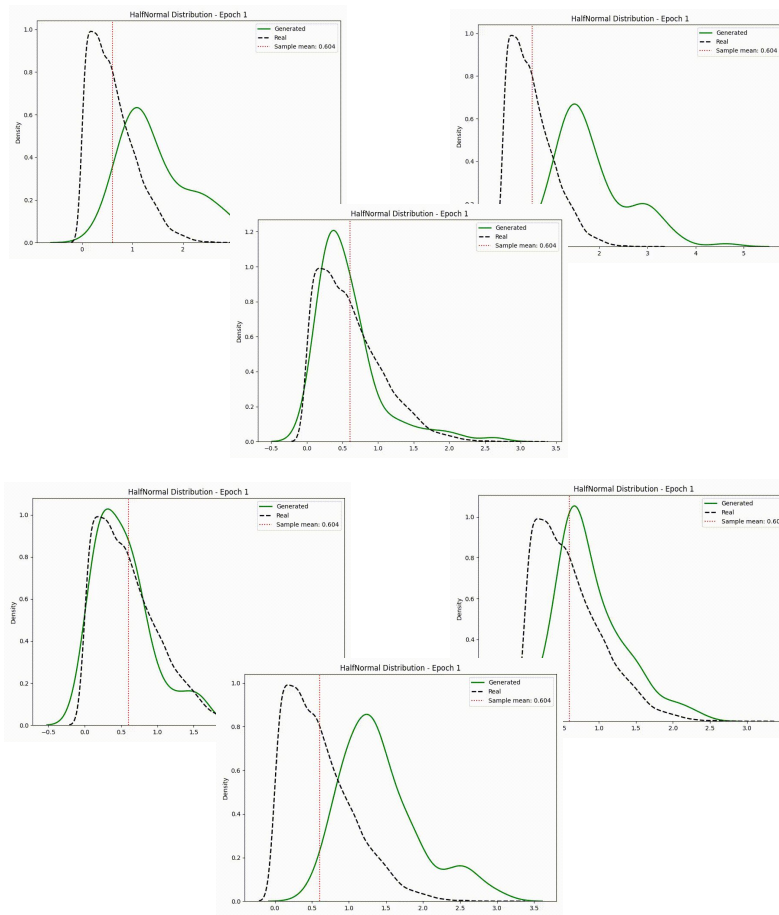
Hyperparameter Analysis

Network Capacity

- In the animations it can be seen that the bottom figure, being the one where the **discriminator is stronger than the generator**, is the **only one that achieves to approximate the small curve at the top**. The reason why the discriminator being slightly stronger helps the generator is discussed in the following sweep.

d_repeats

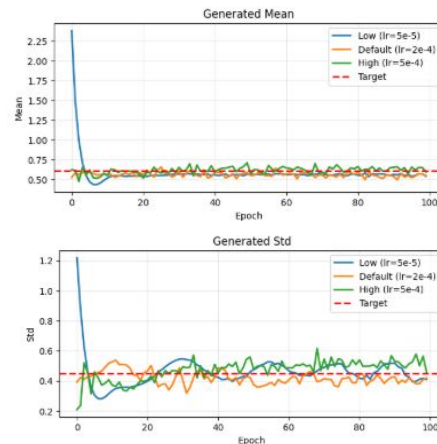
- Again, the mid term is the best ($d_{rep}=3$). The reason for this is probably that as we **increase the repetitions of the discriminator** it gives a **better loss** to the generator regarding its generated images, but **to a point**, as if the **discriminator completely dominates** at the start ($d_{rep}=5$), the generator gets stuck (Vanishing gradients).



Hyperparameter Analysis

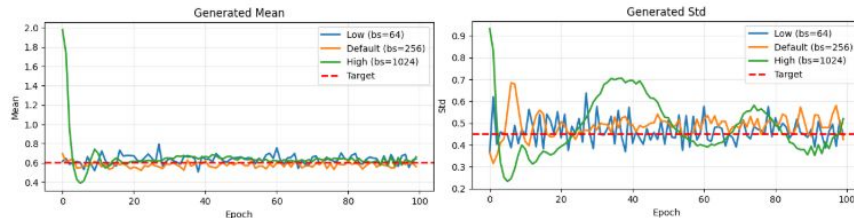
Learning rate

- Given the somewhat easy form of the HalfNormal distribution, a **change in learning rate doesn't really affect** that much the final approximated distributions. Nevertheless, in the charts showing the changes in **approximated mean and std** during the training, it can be seen that the **lower lr makes slower changes** while the **higher lr seems more erratic** in its changes.

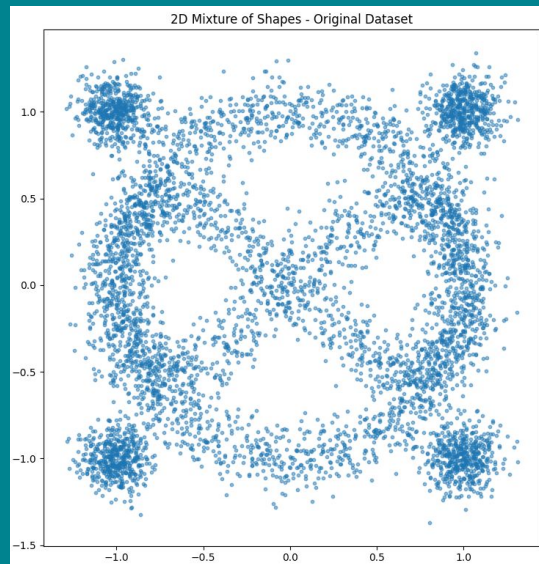


Batch size

- Inverse to the learning rate**, as batch size increases changes become less erratic.



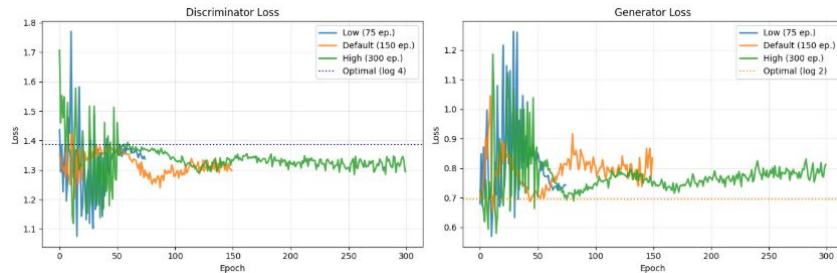
2D Mixture of Shapes



Hyperparameter Analysis

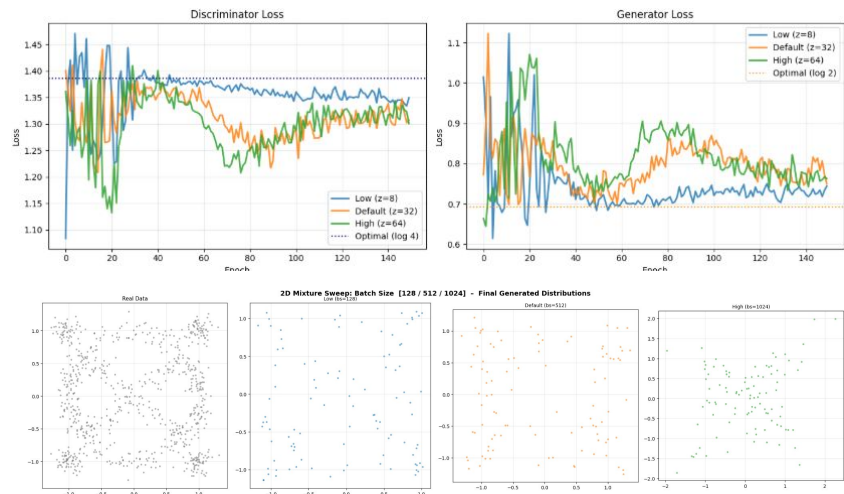
Number of epochs

- Same as in the 1D data, low epochs give the worst results, increasing epochs improves performance up to a certain threshold where the model gets stuck.



Latent Dimension

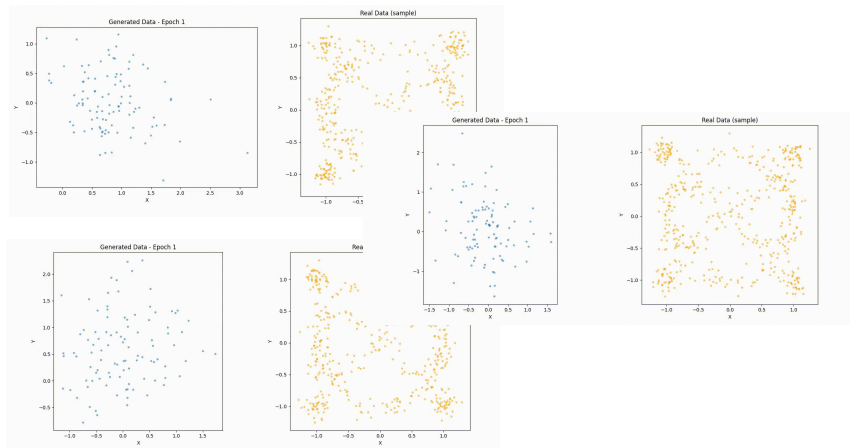
- Looking at the loss plots it seems as if the lowest latent dimension lies closer to both the optimal generator and discriminator loss. Nevertheless, we have already demonstrated in this project that the optimal loss doesn't represent optimal generation. This is demonstrated in the example generation, where the three examples of generation don't seem to represent the distribution that well.



Hyperparameter Analysis

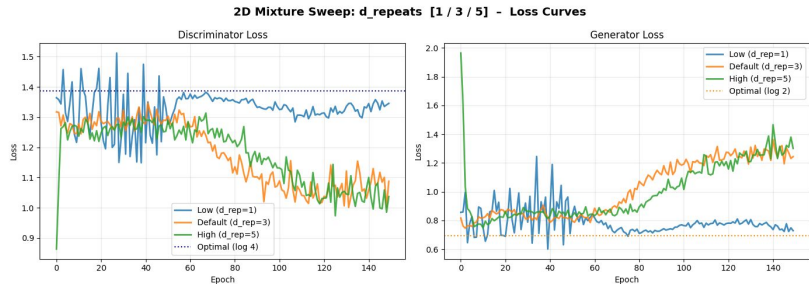
Network Capacity

- The **improved discriminator** over the generator **doesn't seem to help** in the case of 2D data. This can be seen in the animations, where the **equivalent generator and discriminator** (top-left and bottom-left) **start to form the central X** of points **while in the case of the Discriminator being stronger** (right) **than the Generator the latter seems to get stuck** and form blobs of a distribution.



d_repeats

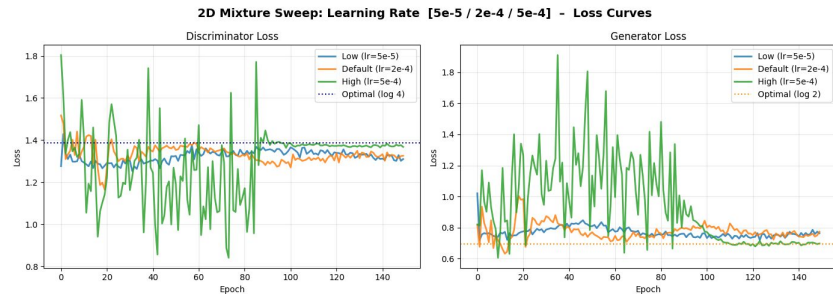
- Same as in the latter experiment**, equivalent discriminator-generator works theoretically best in this case, having the losses closest to the optimal values.



Hyperparameter Analysis

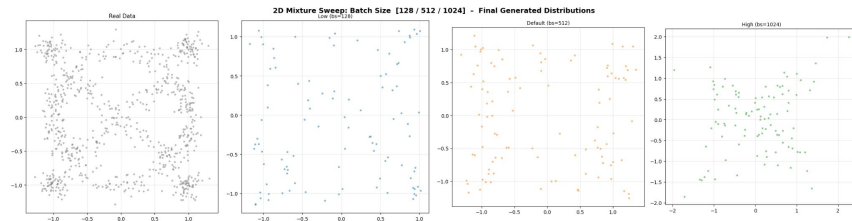
Learning rate

- Same as in the 1D case, the higher the learning rate the more sporadic the training, while lower learning rate provides more stability.



Batch size

- Lowering the batch size provides the best qualitative result from this 2D case. Our hypothesis is that the lower batch size guides the training in a more stochastic way, therefore promoting feature space exploration.



C5: Generative Adversarial Networks

Marina Rosell Murillo

NIU: 1779181

marina.rosellmurillo@autonoma.cat

Gerard Asbert Marcos

NIU: 1603295

gerard.asbert@autonoma.cat

